Chat history

PROMPT :

You are a senior Python game developer and expert Pygame engineer acting as my pair-programming partner.

I am working on a terminal-launched Pygame endless runner project (`SETAPESU26/39_endless-runner`).

Your goal is to guide me through completing the 4 required tasks while maintaining clean, modular, Object-Oriented Code (OOP) structure in accordance with the existing folder layout.

Technical Constraints & Standards

1. Language/Framework: Python 3.10+ using Pygame.

2. Architecture: Preserve and align with the existing modular architecture:

- `main.py`: Entry point, game loop lifecycle, state management (Start/Difficulty, Playing, Game Over).

- `game/player.py`: Player movement, gravity, jump mechanics, and bounding box rects.

- `game/obstacle.py`: Obstacle spawning, scrolling, speed scaling, and bounding box rects.

- `game/game_engine.py`: Core Pygame loop, collision detection, sound triggering, score tracking, rendering.

3. Code Quality:

- Provide minimal, clean, drop-in replacement code blocks.

- Include clear inline comments explaining modified logic.

- Use fallback sound generation (e.g., synthetically generated Pygame sounds via numpy/pygame arrays or dummy sound handlers) so the game never crashes if audio files are missing.

Tasks to Complete

Task 1: Refine Collision Detection & Speed Ramp

- Fix continuous tunneling/collision-skipping bugs at high speeds (e.g., frame-rate independent sweep checks, delta-time physics, capped maximum scroll speed, or standard Pygame `colliderect` continuous hitboxes).

- Cap or smooth the obstacle speed ramp so gameplay remains fair and physically solvable across long runs.

Task 2: Implement Game Over Screen

- On collision, transition the game state to a dedicated "Game Over" screen.

- Render the final score directly on the Pygame display (not just in terminal/console stdout).

- Gracefully pause and wait for user input.

Task 3: Difficulty Selection & Replay Loop

- Provide an interactive restart mechanism from the Game Over screen.

- Implement a Difficulty Selection menu with 3 presets:

  * **Easy:** Low initial speed, slower speed increment, wider spawn intervals.

  * **Medium:** Balanced initial speed and acceleration.

  * **Hard:** High initial speed, rapid acceleration, tighter spawn intervals.

- Ensure all game state variables (score, obstacle list, player position, current speed) reset completely on restart.

Task 4: Audio Feedback System

- Integrate non-blocking Pygame sound effects (`pygame.mixer`) for:

  1. **Jump:** Triggered on jump keypress (Space, Up, W).

  2. **Score / Clear:** Triggered when successfully passing an obstacle.

  3. **Game Over:** Triggered on collision hit.

- Include a graceful fallback if sound hardware/mixer fails or files aren't found.

Interaction Protocol

For every solution or response you provide:

1. Explain the architectural strategy briefly before giving code.

2. Provide the modified files clearly labeled with file path headers (e.g., `### game/game_engine.py`).

3. Point out specific changes made to fix bugs or implement features so I can critically review the code for my assignment submission.

4. Keep solutions step-by-step so I can record clear "Before" and "After" gameplay clips for my submission checklist.

RESPONSE :

Architecture

I kept the original four-module OOP structure:

- **main.py → application state machine: Difficulty → Playing → Game Over**
- **game/player.py → player physics and jump**
- **game/obstacle.py → obstacle movement/lifecycle**
- **game/game_engine.py → simulation, swept collision, scoring, rendering, audio**

The original engine had the main tunneling problem because obstacles moved first and collision was checked only against their final rectangle; the source comments explicitly identify this as the high-speed failure point.

I also changed the original frame-based player physics (vy += 0.8, y += vy) to elapsed-time physics.

Task 1 — Collision + Speed Ramp

Changes:

- **dt-based movement.**
- **Speed is now in pixels/second.**
- **Acceleration is time-based.**
- **Maximum speed is capped per difficulty.**
- **Added swept AABB collision, so an obstacle cannot simply jump from one side of the player to the other between frames.**
- **Frame time is capped at 50 ms in main.py, preventing huge physics jumps after a window pause.**

Task 2 — Game Over

Instead of the previous terminal-only print("Game over..."), the game now changes application state and renders:

GAME OVER
Final Score: X
Press R to replay | Press D for difficulty menu | ESC to quit

The original code only printed the score from render().

Task 3 — Difficulty + Replay

Three presets are implemented:

Difficulty	Initial Speed	Acceleration	Max Speed	Spawn
Easy	300 px/s	7	440	1.35 s
Medium	360 px/s	11	540	1.15 s
Hard	430 px/s	18	660	0.95 s

reset() completely recreates the run state: player, speed, timer, obstacles, distance, score and game-over state.

Task 4 — Audio

Three non-blocking effects are generated programmatically:

- **Jump → short high tone**
- **Score → short higher tone**
- **Game Over → low tone**

No .wav files are required. If the mixer/audio hardware fails, the sound objects become None and the game continues normally.

The original project had no audio system; jump was directly forwarded to player.jump().

---

game/player.py

import pygame

```
class Player:
    """Player movement and physics. Motion is time-based for FPS independence."""

    def __init__(self, x, ground_y, width=30, height=40):
        self.x = float(x)
        self.ground_y = float(ground_y)
        self.width = width
        self.height = height

        # Physics values are expressed per second / per second squared.
        self.gravity = 2200.0
        self.jump_strength = -750.0

        self.y = self.ground_y - self.height
        self.vy = 0.0
        self.on_ground = True

    def reset(self):
        """Return the player to its initial state."""
        self.y = self.ground_y - self.height
        self.vy = 0.0
        self.on_ground = True

    def jump(self):
```

```
    if self.on_ground:
        self.vy = self.jump_strength
        self.on_ground = False
        return True

    return False

def update(self, dt):
    """Advance vertical physics using elapsed time."""
    self.vy += self.gravity * dt
    self.y += self.vy * dt

    ground_level = self.ground_y - self.height

    if self.y >= ground_level:
        self.y = ground_level
        self.vy = 0.0
        self.on_ground = True

def rect(self):
    return pygame.Rect(
        round(self.x),
        round(self.y),
        self.width,
        self.height,
```

)

---

game/obstacle.py

The original obstacle stored speed and moved directly with self.x -= self.speed.

It is now time-based.

```
import pygame



class Obstacle:
    """A horizontally scrolling obstacle."""

    def __init__(self, x, ground_y, speed, width=25, height=40):
        self.x = float(x)
        self.width = width
        self.height = height
        self.y = float(ground_y - height)
        self.speed = float(speed)
        self.scored = False

    def move(self, dt):
        """Move using speed in pixels/second."""
        self.x -= self.speed * dt

    def off_screen(self):
```

```
        return self.x + self.width < 0

    def rect(self):
        return pygame.Rect(
            round(self.x),
            round(self.y),
            self.width,
            self.height,
        )
```

---

game/game_engine.py

This is the main implementation for Tasks 1 and 4.

```
import math
import pygame
from array import array

from .player import Player
from .obstacle import Obstacle



WHITE = (255, 255, 255)
BLACK = (20, 20, 20)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)
RED = (190, 50, 50)
```

```
# Speed is pixels/second.
# Maximum speed prevents the game from becoming physically unsolvable.
DIFFICULTIES = {
    "Easy": {
        "initial_speed": 300.0,
        "acceleration": 7.0,
        "max_speed": 440.0,
        "spawn_interval": 1.35,
    },
    "Medium": {
        "initial_speed": 360.0,
        "acceleration": 11.0,
        "max_speed": 540.0,
        "spawn_interval": 1.15,
    },
    "Hard": {
        "initial_speed": 430.0,
        "acceleration": 18.0,
        "max_speed": 660.0,
        "spawn_interval": 0.95,
    },
}
```

```
class _SoundManager:
    """Non-blocking sound effects with generated fallbacks."""

    def __init__(self):
        self.jump = None
        self.score = None
        self.game_over = None

        try:
            if not pygame.mixer.get_init():
                pygame.mixer.init()

            # No external audio files are required.
            self.jump = self._tone(620, 0.08)
            self.score = self._tone(900, 0.07)
            self.game_over = self._tone(180, 0.22)

        except (pygame.error, ValueError, TypeError):
            # Audio failure must never stop gameplay.
            self.jump = None
            self.score = None
            self.game_over = None
```

```
@staticmethod
def _tone(frequency, duration, volume=0.18):
    mixer_info = pygame.mixer.get_init()

    if not mixer_info:
        return None

    sample_rate = mixer_info[0]
    sample_count = max(1, int(sample_rate * duration))
    amplitude = int(32767 * volume)

    samples = array("h")

    for i in range(sample_count):
        envelope = 1.0 - (i / sample_count)

        value = int(
            amplitude
            * envelope
            * math.sin(
                2.0 * math.pi * frequency * i / sample_rate
            )
        )

        samples.append(value)
```

```
        return pygame.mixer.Sound(buffer=samples.tobytes())

    @staticmethod
    def play(sound):
        if sound is not None:
            try:
                sound.play()
            except pygame.error:
                pass


class GameEngine:
    """Core simulation for one endless-runner session."""

    def __init__(self, width, height, difficulty="Medium"):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.font = pygame.font.SysFont("Arial", 30)
        self.large_font = pygame.font.SysFont(
            "Arial",
            54,
            bold=True,
```

```
    )
    self.small_font = pygame.font.SysFont("Arial", 22)

    self.sounds = _SoundManager()
    self.difficulty = difficulty

    self.reset(difficulty)

def reset(self, difficulty=None):
    """Completely reset one gameplay session."""

    if difficulty is not None:
        self.difficulty = difficulty

    settings = DIFFICULTIES[self.difficulty]

    # Player state.
    self.player = Player(80, self.ground_y)

    # Difficulty state.
    self.speed = settings["initial_speed"]
    self.acceleration = settings["acceleration"]
    self.max_speed = settings["max_speed"]
    self.spawn_interval = settings["spawn_interval"]
```

```
    # Run state.
    self._spawn_timer = 0.0
    self.obstacles = []
    self.distance = 0.0
    self.score = 0
    self.game_over = False

def handle_event(self, event):
    """Handle gameplay-specific input."""

    if (
        event.type == pygame.KEYDOWN
        and event.key
        in (
            pygame.K_SPACE,
            pygame.K_UP,
            pygame.K_w,
        )
        and not self.game_over
    ):
        if self.player.jump():
            self.sounds.play(self.sounds.jump)

def handle_input(self):
    # Kept for compatibility with the original architecture.
```

```
        pass

    @staticmethod
    def _swept_aabb(
        player_old,
        player_new,
        obstacle_old,
        obstacle_new,
    ):
        """
        Continuous collision detection.

        Treat the player's top-left point as moving relative to an expanded
        obstacle. This detects a collision anywhere during the frame instead
        of checking only the final rectangles.
        """

        # Already overlapping at the beginning of the frame.
        if player_old.colliderect(obstacle_old):
            return True

        # Player movement relative to obstacle movement.
        dx = (
            (player_new.x - player_old.x)
            - (obstacle_new.x - obstacle_old.x)
```

```
)

dy = (
    (player_new.y - player_old.y)
    - (obstacle_new.y - obstacle_old.y)
)

# Expand obstacle by the player's width/height.
expanded_left = obstacle_old.x - player_old.width
expanded_right = obstacle_old.x + obstacle_old.width

expanded_top = obstacle_old.y - player_old.height
expanded_bottom = obstacle_old.y + obstacle_old.height

# Player's starting top-left point.
px = player_old.x
py = player_old.y

def axis_entry_exit(
    position,
    delta,
    low,
    high,
):
    if delta == 0:
```

```
        if low <= position <= high:
            return -math.inf, math.inf

        return math.inf, -math.inf

    t1 = (low - position) / delta
    t2 = (high - position) / delta

    return min(t1, t2), max(t1, t2)

x_entry, x_exit = axis_entry_exit(
    px,
    dx,
    expanded_left,
    expanded_right,
)

y_entry, y_exit = axis_entry_exit(
    py,
    dy,
    expanded_top,
    expanded_bottom,
)

entry = max(
```

```
            x_entry,
            y_entry,
            0.0,
        )

        exit_ = min(
            x_exit,
            y_exit,
            1.0,
        )

        return entry <= exit_

    def update(self, dt):
        if self.game_over:
            return

        # Smooth acceleration with a hard upper limit.
        self.speed = min(
            self.max_speed,
            self.speed + self.acceleration * dt,
        )

        # Save old player rectangle for continuous collision detection.
        previous_player_rect = self.player.rect()
```

```
self.player.update(dt)

current_player_rect = self.player.rect()

# Time-based obstacle spawning.
self._spawn_timer += dt

if self._spawn_timer >= self.spawn_interval:
    self._spawn_timer -= self.spawn_interval

    self.obstacles.append(
        Obstacle(
            self.width,
            self.ground_y,
            self.speed,
        )
    )

# Move obstacles and test the complete movement path.
for obstacle in self.obstacles:
    previous_obstacle_rect = obstacle.rect()

    obstacle.speed = self.speed
    obstacle.move(dt)
```

```
        current_obstacle_rect = obstacle.rect()

        if self._swept_aabb(
            previous_player_rect,
            current_player_rect,
            previous_obstacle_rect,
            current_obstacle_rect,
        ):
            self.game_over = True
            self.sounds.play(self.sounds.game_over)
            return

    # Award one point for every successfully cleared obstacle.
    for obstacle in self.obstacles:
        if (
            not obstacle.scored
            and obstacle.x + obstacle.width < self.player.x
        ):
            obstacle.scored = True
            self.score += 1

            self.sounds.play(self.sounds.score)

    # Remove obstacles that have completely left the screen.
```

```
    self.obstacles = [
        obstacle
        for obstacle in self.obstacles
        if not obstacle.off_screen()
    ]

    self.distance += self.speed * dt

def render(self, screen):
    pygame.draw.line(
        screen,
        BROWN,
        (0, self.ground_y),
        (self.width, self.ground_y),
        4,
    )

    pygame.draw.rect(
        screen,
        WHITE,
        self.player.rect(),
    )

    for obstacle in self.obstacles:
        pygame.draw.rect(
```

```
            screen,
            DARK_GREEN,
            obstacle.rect(),
        )

    score_text = self.font.render(
        f"Score: {self.score}   Speed: {self.speed:.0f}",
        True,
        BLACK,
    )

    screen.blit(
        score_text,
        (10, 10),
    )

    difficulty_text = self.small_font.render(
        f"Difficulty: {self.difficulty}",
        True,
        BLACK,
    )

    screen.blit(
        difficulty_text,
        (10, 45),
```

```
    )

def render_game_over(self, screen):
    """Render the dedicated game-over screen."""

    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA,
    )

    overlay.fill(
        (0, 0, 0, 165)
    )

    screen.blit(
        overlay,
        (0, 0),
    )

    title = self.large_font.render(
        "GAME OVER",
        True,
        RED,
    )
```

```
score = self.font.render(
    f"Final Score: {self.score}",
    True,
    WHITE,
)

prompt = self.small_font.render(
    "Press R to replay  |  Press D for difficulty menu  |  ESC to quit",
    True,
    WHITE,
)

screen.blit(
    title,
    title.get_rect(
        center=(self.width // 2, 130)
    ),
)

screen.blit(
    score,
    score.get_rect(
        center=(self.width // 2, 205)
    ),
)
```

```
        screen.blit(
            prompt,
            prompt.get_rect(
                center=(self.width // 2, 270)
            ),
        )
```

---

main.py

The original main.py had one permanent GameEngine and continuously called update()/render().

This version introduces the required state machine.

```
import pygame

from game.game_engine import DIFFICULTIES, GameEngine



pygame.init()



WIDTH, HEIGHT = 800, 400

SCREEN = pygame.display.set_mode(
    (WIDTH, HEIGHT)
)
```

```
pygame.display.set_caption(
    "Endless Runner - Pygame Version"
)



SKY = (200, 220, 240)
BLACK = (20, 20, 20)
WHITE = (255, 255, 255)

clock = pygame.time.Clock()
FPS = 60



class Game:
    """Application-level state machine."""

    START = "start"
    PLAYING = "playing"
    GAME_OVER = "game_over"

    def __init__(self):
        self.engine = GameEngine(
            WIDTH,
            HEIGHT,
```

```
    "Medium",
)

self.state = self.START

self.selected_index = 1
self.difficulties = list(
    DIFFICULTIES.keys()
)

self.title_font = pygame.font.SysFont(
    "Arial",
    48,
    bold=True,
)

self.font = pygame.font.SysFont(
    "Arial",
    28,
)

self.small_font = pygame.font.SysFont(
    "Arial",
    20,
)
```

```
@property
def selected_difficulty(self):
    return self.difficulties[
        self.selected_index
    ]

def start_run(self):
    # Reset every gameplay variable before replaying.
    self.engine.reset(
        self.selected_difficulty
    )

    self.state = self.PLAYING

def handle_event(self, event):
    if event.type == pygame.QUIT:
        return False

    # -------------------------
    # Difficulty selection state
    # -------------------------
    if self.state == self.START:

        if event.type == pygame.KEYDOWN:
```

```
if event.key in (
    pygame.K_UP,
    pygame.K_w,
):
    self.selected_index = (
        self.selected_index - 1
    ) % len(self.difficulties)

elif event.key in (
    pygame.K_DOWN,
    pygame.K_s,
):
    self.selected_index = (
        self.selected_index + 1
    ) % len(self.difficulties)

elif event.key in (
    pygame.K_RETURN,
    pygame.K_SPACE,
):
    self.start_run()

elif event.key == pygame.K_ESCAPE:
    return False
```

```
# -------------------------
# Active gameplay state
# -------------------------
elif self.state == self.PLAYING:

    self.engine.handle_event(
        event
    )

# -------------------------
# Game-over state
# -------------------------
elif self.state == self.GAME_OVER:

    if event.type == pygame.KEYDOWN:

        if event.key == pygame.K_r:
            # Replay same difficulty.
            self.start_run()

        elif event.key == pygame.K_d:
            # Return to difficulty selection.
            self.state = self.START
```

```
            elif event.key == pygame.K_ESCAPE:
                return False

        return True

    def update(self, dt):
        if self.state == self.PLAYING:

            self.engine.update(dt)

            if self.engine.game_over:
                self.state = self.GAME_OVER

    def render_start(self):
        SCREEN.fill(SKY)

        title = self.title_font.render(
            "ENDLESS RUNNER",
            True,
            BLACK,
        )

        subtitle = self.font.render(
            "Select Difficulty",
            True,
```

```
    BLACK,
)

SCREEN.blit(
    title,
    title.get_rect(
        center=(WIDTH // 2, 70)
    ),
)

SCREEN.blit(
    subtitle,
    subtitle.get_rect(
        center=(WIDTH // 2, 125)
    ),
)

for index, difficulty in enumerate(
    self.difficulties
):
    selected = (
        index == self.selected_index
    )

    text = self.font.render(
```

```
            f"{'>' if selected else ' '} {difficulty}",
            True,
            (30, 100, 30)
            if selected
            else BLACK,
        )

        SCREEN.blit(
            text,
            text.get_rect(
                center=(
                    WIDTH // 2,
                    185 + index * 50,
                )
            ),
        )

    controls = self.small_font.render(
        "UP/DOWN or W/S: select    "
        "ENTER/SPACE: start    ESC: quit",
        True,
        BLACK,
    )

    SCREEN.blit(
```

```
            controls,
            controls.get_rect(
                center=(WIDTH // 2, 360)
            ),
        )

    def render(self):
        if self.state == self.START:
            self.render_start()
            return

        SCREEN.fill(SKY)

        self.engine.render(
            SCREEN
        )

        if self.state == self.GAME_OVER:
            self.engine.render_game_over(
                SCREEN
            )


def main():
    game = Game()
```

```
running = True

while running:

    # Convert milliseconds to seconds.
    # Cap long pauses to avoid huge physics jumps.
    dt = min(
        clock.tick(FPS) / 1000.0,
        0.05,
    )

    for event in pygame.event.get():

        running = game.handle_event(
            event
        )

        if not running:
            break

    if not running:
        break

    game.update(dt)
```

```
        game.render()

        pygame.display.flip()

    pygame.quit()



if __name__ == "__main__":
    main()
```

What you should demonstrate in your submission

Clip 1 — Difficulty Selection

Launch:

python main.py

Show:

ENDLESS RUNNER

Select Difficulty

> Easy

Medium

Hard

Use ↑/↓ or W/S, then Enter.

Clip 2 — Jump + Audio

During gameplay:

- **Space**
- **↑**

- **W**

all trigger the jump.

The jump sound is generated at runtime, so you don't need to submit external audio files.

Clip 3 — Score + Clear

Allow an obstacle to pass underneath/behind the player.

You should see:

Score: 1

and hear the score-clear sound.

Clip 4 — Collision

Allow the player to hit an obstacle.

The gameplay should stop and transition to:

GAME OVER

Final Score: X

Press R to replay | Press D for difficulty menu | ESC to quit

Clip 5 — Replay

Press R.

Verify:

Score = 0

obstacles = empty

player = starting position

speed = difficulty's initial speed

game_over = False

Clip 6 — Difficulty Reset

From Game Over press D, select another difficulty, and start again.

This demonstrates that the difficulty state and all gameplay variables are being reset rather than carrying values from the previous run.

One important implementation detail for your viva

If asked "How did you solve collision tunneling?", don't simply say colliderect().

Your answer should be:

"I implemented continuous swept-AABB collision detection. Instead of testing only the obstacle's final rectangle after each frame, I compare the previous and current positions and test the relative movement against an expanded obstacle hitbox. Therefore, even if an obstacle crosses the player's complete width within one frame, the crossing is detected."

That is materially different from the original final-position-only check.

I also prepared the complete replacement project as a ZIP:

Download the completed Endless Runner solution

The original uploaded files remain untouched; the ZIP contains the replacement main.py plus game/player.py, game/obstacle.py, and game/game_engine.py.